



PROJETO E ANÁLISE DE ALGORITMOS

AULA 5 – NOTAÇÃO ASSINTÓTICA

5.1 NOTAÇÃO O, NOTAÇÃO Ω E NOTAÇÃO Θ

- Quando analisamos o tempo de execução da ordenação por inserção para o pior caso, começamos com a **complicada expressão** que representava de maneira **detalhada** o custo das operações do procedimento.
- Depois, **descartamos** os termos de mais baixa ordem e também ignoramos os coeficientes que apareciam na equação. Com isso, restou apenas o fator n^2 , que colocamos na notação Θ como $\Theta(n^2)$.

5.1 NOTAÇÃO O, NOTAÇÃO Ω E NOTAÇÃO Θ

$$\begin{aligned} T(n) &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8) \end{aligned}$$

$$T(n) = an^2 + bn + c = \theta(n^2)$$

- Esse estilo foi utilizado para caracterizar os tempos de execução dos algoritmos: descartar os termos de mais baixa ordem e o coeficiente do termo inicial, usando uma notação que enfatiza a taxa de crescimento do tempo de execução.

5.1 NOTAÇÃO O, NOTAÇÃO Ω E NOTAÇÃO Θ

- A **notação O** caracteriza um limite superior sobre o comportamento assintótico de uma função.
- Em outras palavras, diz que uma função não cresce mais rápido do que uma certa taxa, com base no termo de mais alta ordem.
- Por exemplo, considere a função $7n^3 + 100n^2 - 20n + 6$. Seu termo de mais alta ordem é $7n^3$, e com isso dizemos que a taxa de crescimento dessa função é n^3 .

5.1 NOTAÇÃO O, NOTAÇÃO Ω E NOTAÇÃO Θ

- Como essa função não cresce mais rápido que n^3 , podemos escrever que ela é $O(n^3)$.
- Essa função também é $O(n^4)$. Como a função cresce mais lentamente que n^4 , está correto afirmar que ela não cresce mais rápido.
- Essa função também é $O(n^5)$, $O(n^6)$ e assim por diante. De um modo geral, ela é $O(n^c)$ para qualquer constante $c \geq 3$.

5.1 NOTAÇÃO O, NOTAÇÃO Ω E NOTAÇÃO Θ

- A **notação Ω** caracteriza um limite inferior sobre o comportamento assintótico de uma função.
- Em outras palavras, ela diz que uma função cresce pelo menos tão rapidamente quanto uma certa taxa, com base — assim como na notação O — no termo de mais alta ordem. Como o termo de mais alta ordem na função $7n^3 + 100n^2 - 20n + 6$ cresce pelo menos tão rapidamente quanto n^3 , essa função é $\Omega(n^3)$. Essa função também é $\Omega(n^2)$ e $\Omega(n)$. De modo geral, ela é $\Omega(n^c)$ para qualquer constante $c \leq 3$.

5.1 NOTAÇÃO O , NOTAÇÃO Ω E NOTAÇÃO Θ

- A **notação Θ** caracteriza um limite restrito sobre o comportamento assintótico de uma função. Ela diz que uma função cresce precisamente em determinada taxa, baseada no termo de mais alta ordem.
- Em outras palavras, a notação Θ caracteriza a taxa de crescimento da função para dentro de um fator constante limitado **acima e abaixo**.
- Esses dois fatores constantes **não precisam ser iguais**.

5.1 NOTAÇÃO O, NOTAÇÃO Ω E NOTAÇÃO Θ

- Se pudermos mostrar que uma função é tanto $O(f(n))$ quanto $\Omega(f(n))$ para alguma função $f(n)$, então podemos demonstrar que a função é $\Theta(f(n))$.
- Por exemplo, visto que a função $7n^3 + 100n^2 - 20n + 6$ é tanto $O(n^3)$ quanto $\Omega(n^3)$, ela também é $\Theta(n^3)$.

5.2 DEFINIÇÕES FORMAIS

- As notações que usamos para descrever o tempo de execução assintótico de um algoritmo são definidas em termos de funções cujos domínios são tipicamente o conjunto $\mathbb{N}$ de números naturais ou o conjunto $\mathbb{R}$ de números reais.

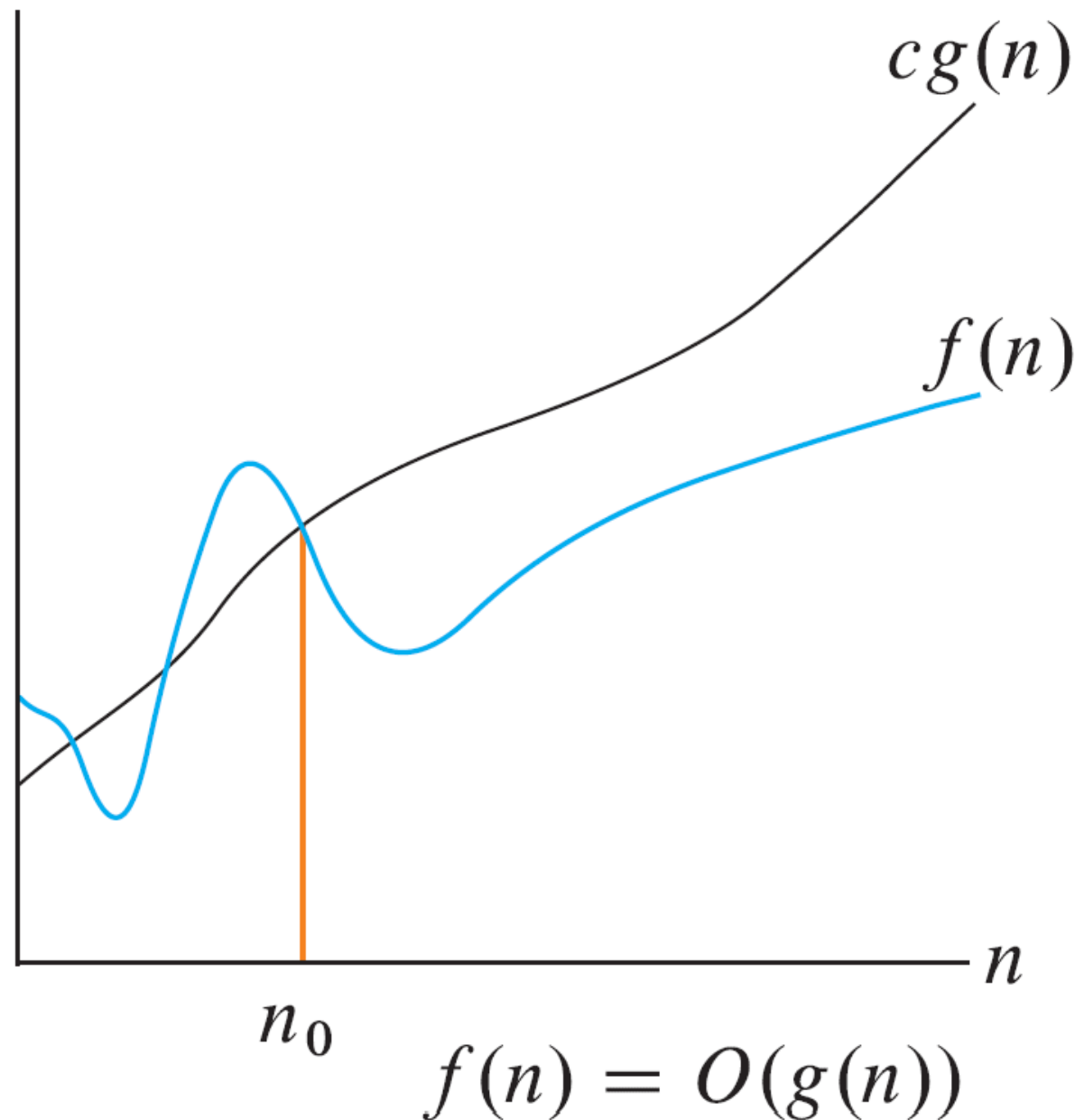
5.2 DEFINIÇÕES FORMAIS

- A **notação O** descreve um limite assintótico superior. Usamos a notação O para oferecer limite superior em uma função, dentro de um fator constante.
- Definição formal: para determinada função $g(n)$, indicamos com $O(g(n))$ o conjunto de funções:

$O(g(n)) = \{f(n) : \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que } 0 \leq f(n) \leq cg(n) \text{ para todo } n \geq n_0\}.$

5.2 DEFINIÇÕES FORMAIS

- Uma função $f(n)$ pertencerá ao conjunto $O(g(n))$ se houver uma constante positiva c tal que $f(n) \leq cg(n)$ para um n suficientemente grande.
- Para todos os valores n na posição e à direita de n_0 , o valor da função $f(n)$ está na altura de $cg(n)$ ou abaixo.



5.2 DEFINIÇÕES FORMAIS

- A definição de $O(g(n))$ requer que cada função $f(n)$ no conjunto $O(g(n))$ seja assintoticamente **não negativa**: $f(n)$ precisa ser não negativo sempre que n for suficientemente grande.
 - Uma função assintoticamente positiva é aquela que é positiva para todo n suficientemente grande.
- Consequentemente, a própria função $g(n)$ precisa ser assintoticamente não negativa, ou então o conjunto $O(g(n))$ estará vazio.

5.2 DEFINIÇÕES FORMAIS

- Vamos provar que:

$$4n^2 + 100n + 500 = O(n^2)$$

- Para isso, precisamos encontrar constantes positivas c e n_0 tais que:

$$4n^2 + 100n + 500 \leq cn^2 \quad \forall n \geq n_0$$

- Dividindo ambos os lados por n^2 , temos:

$$4 + \frac{100}{n} + \frac{500}{n^2} \geq c$$

- Essa equação é satisfeita por muitas escolhas de c e n_0 .

5.2 DEFINIÇÕES FORMAIS

$$4 + \frac{100}{n} + \frac{500}{n^2} \geq c$$

- Por exemplo, se escolhermos $n_0 = 1$, então essa inequação é verdadeira para $c = 604$. Se escolhermos $n_0 = 10$, então $c = 19$ funciona, e a escolha de $n_0 = 100$ nos permite usar $c = 5,05$.

5.2 DEFINIÇÕES FORMAIS

- Também podemos usar a definição formal da notação O para mostrar que a função $n^3 - 100n^2$ não pertence ao conjunto $O(n^2)$, embora o coeficiente de n^2 seja um número negativo grande.
- Se tivéssemos $n^3 - 100n^2 = O(n^2)$, então haveria constantes positivas c e n_0 tais que $n^3 - 100n^2 \leq cn^2$ para todo $n \geq n_0$.
- Novamente, dividimos os dois lados por n^2 , resultando em $n - 100 \leq c$. Não importando o valor que tenhamos escolhido para a constante c , essa inequação não é verdadeira para qualquer valor de $n > c + 100$.

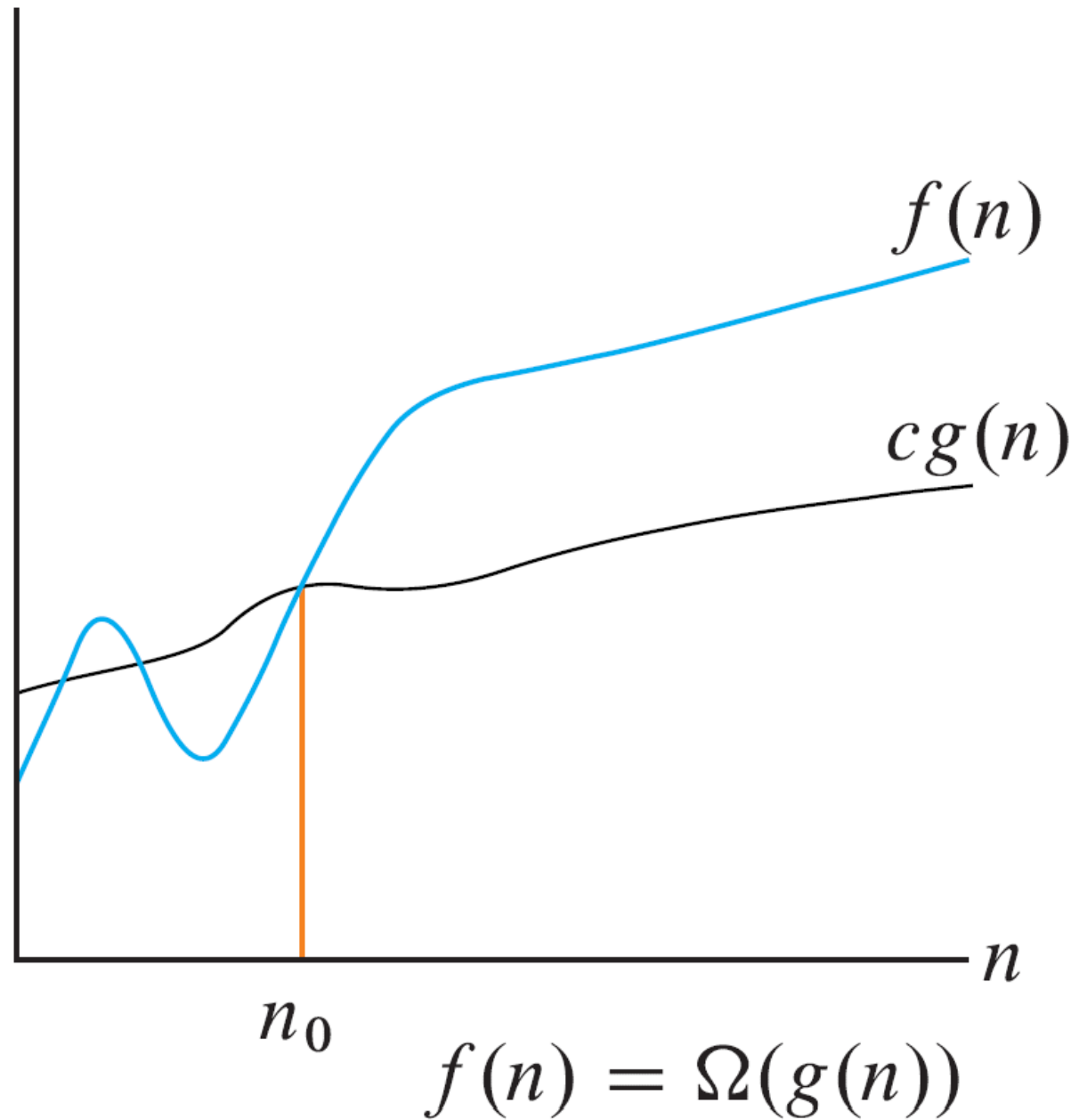
5.2 DEFINIÇÕES FORMAIS

- Assim como a notação O oferece um limite assintótico superior sobre uma função, a **notação Ω** oferece um **limite assintótico inferior**.
- Definição formal:

$\Omega(g(n)) = \{f(n) : \text{existem constantes positivas } c \text{ e } n_0 \text{ tais que } 0 \leq cg(n) \leq f(n) \text{ para todo } n \geq n_0\}$

5.2 DEFINIÇÕES FORMAIS

- Na figura ao lado, para todos os valores de n em n_0 ou à sua direita, o valor de $f(n)$ está em $cg(n)$ ou acima disso.



5.2 DEFINIÇÕES FORMAIS

- Vamos mostrar que

$$4n^2 + 100n + 500 = \Omega(n^2)$$

- Precisamos encontrar constantes positivas c e n_0 tais que $4n^2 + 100n + 500 \geq cn^2$ para todo $n \geq n_0$.
- Dividimos os dois lados por n^2 , resultando em:

$$4 + \frac{100}{n} + \frac{500}{n^2} \geq c$$

- Essa inequação é verdadeira quando n_0 é qualquer inteiro positivo e $c=4$.

5.2 DEFINIÇÕES FORMAIS

- Mesmo com um coeficiente pequeno para o termo n^2 e se o termos de baixa ordem forem negativos, a função ainda seria $\Omega(n^2)$. Por exemplo:

$$\frac{n^2}{100} - 100n - 500 = \Omega(n^2)$$

- Dividimos os dois lados por n^2 , resultando em:

$$\frac{1}{100} - \frac{100}{n} - \frac{500}{n^2} \geq c$$

5.2 DEFINIÇÕES FORMAIS

- Podemos escolher qualquer valor para n_0 que seja pelo menos 10.005 e encontrar um valor positivo para c .
- Por exemplo, quando $n_0 = 10.005$, podemos escolher $c = 2,49 \times 10^{-9}$.
- Se selecionarmos um valor maior para n_0 , também podemos aumentar c . Por exemplo, se $n_0 = 100.000$, então podemos escolher $c = 0,0089$. Quanto maior for o valor de n_0 , mais perto do coeficiente $1/100$ podemos escolher como valor de c .

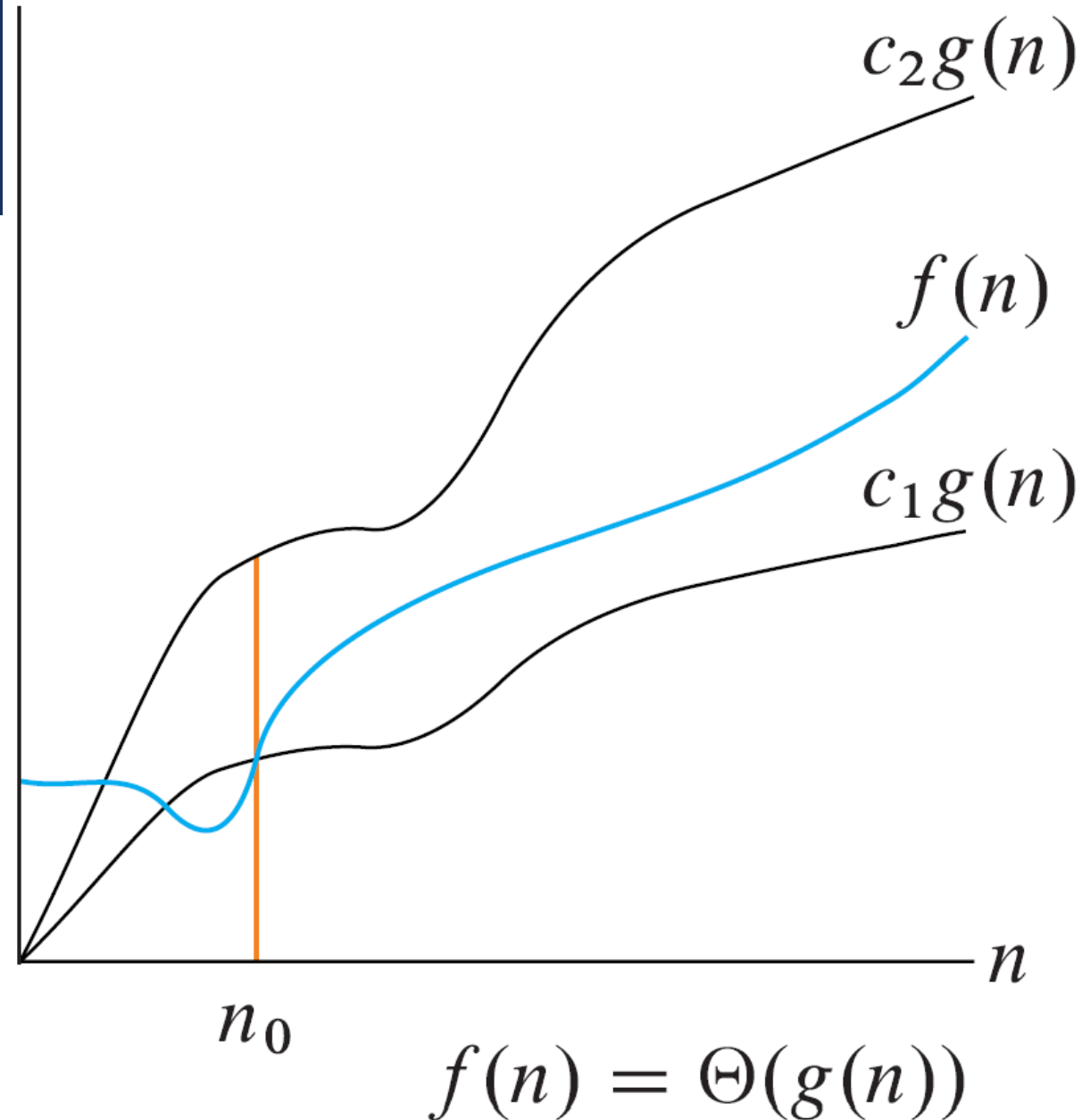
5.2 DEFINIÇÕES FORMAIS

- Usamos a **notação** Θ para limites assintoticamente restritos.
- Definição formal:

$\Theta(g(n)) = \{f(n) : \text{existem constantes positivas } c_1, c_2 \text{ e } n_0 \text{ tais que } 0 \leq c_1g(n) \leq f(n) \leq c_2g(n) \text{ para todo } n \geq n_0\}$

5.2 DEFINIÇÕES FORMAIS

- A figura ao lado mostra a intuição por trás da notação Θ .
- Para todos os valores de n em n_0 ou à sua direita, o valor de $f(n)$ está em $c_1g(n)$ ou acima disso e abaixo de $c_2g(n)$. Em outras palavras, para todo $n \geq n_0$, a função $f(n)$ é igual a $g(n)$, dentro de fatores constantes.



5.2 DEFINIÇÕES FORMAIS

- Teorema: Para quaisquer duas funções $f(n)$ e $g(n)$, temos $f(n) = \Theta(g(n))$ se, e somente se, $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$.
- Quando utilizamos a notação assintótica para caracterizar o tempo de execução de um algoritmo, devemos ter certeza de que a notação assintótica utilizada é a mais precisa possível, sem exagerar o tempo de execução ao qual se aplica.

5.2 DEFINIÇÕES FORMAIS

- Podemos dizer corretamente que o tempo de execução **no pior caso** da ordenação por inserção é $O(n^2)$, $\Omega(n^2)$ e, em razão do teorema anterior, $\Theta(n^2)$.
- Embora todas as três formas de caracterizar os tempos de execução no pior caso estejam corretas, o limite $\Theta(n^2)$ é o **mais preciso** e, portanto, o preferido.
- Também podemos dizer corretamente que o tempo de **execução no melhor caso** da ordenação por inserção é $O(n)$, $\Omega(n)$ e $\Theta(n)$, novamente com $\Theta(n)$ sendo o mais preciso e, portanto, o preferido.

5.2 DEFINIÇÕES FORMAIS

- É incorreto afirmar que o tempo de execução da ordenação por inserção é $\Theta(n^2)$.
- Isso é um exagero porque, ao omitirmos “pior caso” da declaração, ficamos com uma declaração geral cobrindo todos os casos. O erro aqui é que a ordenação por inserção não é executada em tempo $\Theta(n^2)$ em todos os casos, pois, como já vimos, ele é executado em tempo $\Theta(n)$ no melhor caso.

5.2 DEFINIÇÕES FORMAIS

- Contudo, podemos dizer corretamente que o tempo de execução da **ordenação por inserção é $O(n^2)$** porque, em todos os casos, seu tempo de execução não cresce mais rápido que n^2 . Quando dizemos $O(n^2)$ em vez de $\Theta(n^2)$, não há problema em ter casos cujo tempo de execução cresce mais lentamente que n^2 .
- Da mesma forma, não podemos dizer corretamente que o tempo de execução da ordenação por inserção é $\Theta(n)$, mas podemos dizer que seu tempo de execução é $\Omega(n)$.

5.2 DEFINIÇÕES FORMAIS

- O limite assintótico superior fornecido pela notação O pode ser ou não ser assintoticamente justo. O limite $2n^2 = O(n^2)$ é assintoticamente justo, mas o limite $2n = O(n^2)$ não é. Usamos a **notação o** para denotar um limite superior que não é assintoticamente justo.
- Definição formal:
$$o(g(n)) = \{f(n) : \text{para qualquer constante positiva } c > 0, \text{ existe uma constante } n_0 > 0 \text{ tal que } 0 \leq f(n) < cg(n) \text{ para todo } n \leq n_0\}$$
- Por exemplo, $2n = o(n^2)$, mas $2n^2 \neq o(n^2)$.

5.2 DEFINIÇÕES FORMAIS

- As definições da notação O e da notação o são semelhantes. A principal diferença é que, em $f(n) = O(g(n))$, o limite $0 \leq f(n) \leq cg(n)$ se mantém válido para alguma constante $c > 0$ mas, em $f(n) = o(g(n))$, o limite $0 \leq f(n) < cg(n)$ é válido para todas as constantes $c > 0$.
- Por analogia, a notação ω está para a notação Ω como a notação o está para a notação O .
- Usamos a notação ω para indicar um limite inferior que não é assintoticamente justo.

5.2 DEFINIÇÕES FORMAIS

- Definição formal:

$\omega(g(n)) = \{f(n): \text{para qualquer constante positiva } c > 0, \text{ existe uma constante } n_0 > 0 \text{ tal que } 0 \leq cg(n) < f(n) \text{ para todo } n \geq n_0\}$

- Enquanto a definição da notação O diz que $f(n) < cg(n)$, a definição da notação ω diz o oposto: que $cg(n) < f(n)$. Como exemplos da notação ω , temos $n^2/2 = \omega(n)$, mas $n^2/2 \neq \omega(n^2)$.

5.2 EXERCÍCIOS

■ Verifique se:

a) $2n + 5 = O(n)$

b) $n^2 + 2n + 1 = O(n^2)$

c) $7n^2 = O(n^3)$

d) $\frac{3n^2}{2} + \frac{7n}{2} + 4 = O(n^2)$

e) $2n^2 + 30n + 400 = O(n^2)$

f) $6n^3 = \theta(n^2)$

g) $n^2 = O(n)$

h) $\lg(n) = O(n)$

i) $n = O(2^n)$

j) $\frac{n^2}{2} - 100n - 300 = \Omega(n)$

k) $8n^3 + 5n^2 + 7 = \Omega(n^3)$

5.2 EXERCÍCIOS

- Sejam $f(n)$ e $g(n)$ funções assintoticamente não negativas. Usando a definição básica da notação Θ , prove que $\max\{f(n), g(n)\} = \Theta(f(n) + g(n))$.
- Explique por que a declaração “O tempo de execução no algoritmo A é no mínimo $O(n^2)$ ” não tem sentido.
- Prove que $\Theta(f(n)) \cdot \Theta(g(n)) = \Theta(f(n) \cdot g(n))$

5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE

- $f(n) = O(1)$

- Algoritmos de complexidade $O(1)$ são ditos de complexidade constante.
- Uso do algoritmo independe de n .
- As instruções do algoritmo são executadas um número fixo de vezes.

5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE

- $f(n) = O(\log(n))$
 - Um algoritmo de complexidade $O(\log n)$ é dito ter complexidade logarítmica.
 - Típico em algoritmos que transformam um problema em outros menores.
 - Pode-se considerar o tempo de execução como menor que uma constante grande.
 - Quando n é 1000, $\log_2 n \approx 10$, quando n é 1 milhão, $\log_2 n \approx 20$.
 - Para dobrar o valor de $\log n$ temos de considerar o quadrado de n .
 - A base do logaritmo muda pouco estes valores: quando n é 1 milhão, o $\log_2 n$ é 20 e o $\log_{10} n$ é 6.

5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE

- $f(n) = O(n)$
 - Um algoritmo de complexidade $O(n)$ é dito de complexidade linear.
 - Em geral, um pequeno trabalho é realizado sobre cada elemento de entrada.
 - É a melhor situação possível para um algoritmo que tem de processar/produzir n elementos de entrada/saída.
 - Cada vez que n dobra de tamanho, o tempo de execução também dobra.

5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE

- $f(n) = O(n \log n)$
 - Típico em algoritmos que quebram um problema em outros menores, resolvem cada um deles independentemente e juntando as soluções depois.
 - Quando n é 1 milhão, $n \log_2 n$ é cerca de 20 milhões.
 - Quando n é 2 milhões, $n \log_2 n$ é cerca de 42 milhões, pouco mais do que o dobro.

5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE

- $f(n) = O(n^2)$
 - Um algoritmo de complexidade $O(n^2)$ é dito de complexidade quadrática.
 - Ocorrem quando os itens de dados são processados aos pares, muitas vezes em um laço dentro de outro.
 - Quando n é mil, o número de operações é da ordem de 1 milhão.
 - Sempre que n dobra, o tempo de execução é multiplicado por 4.
 - Úteis para resolver problemas de tamanhos relativamente pequenos.

5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE

- $f(n) = O(n^3)$
 - Um algoritmo de complexidade $O(n^3)$ é dito de complexidade cúbica.
 - Úteis apenas para resolver pequenos problemas.
 - Quando n é 100, o número de operações é da ordem de 1 milhão.
 - Sempre que n dobra, o tempo de execução fica multiplicado por 8.

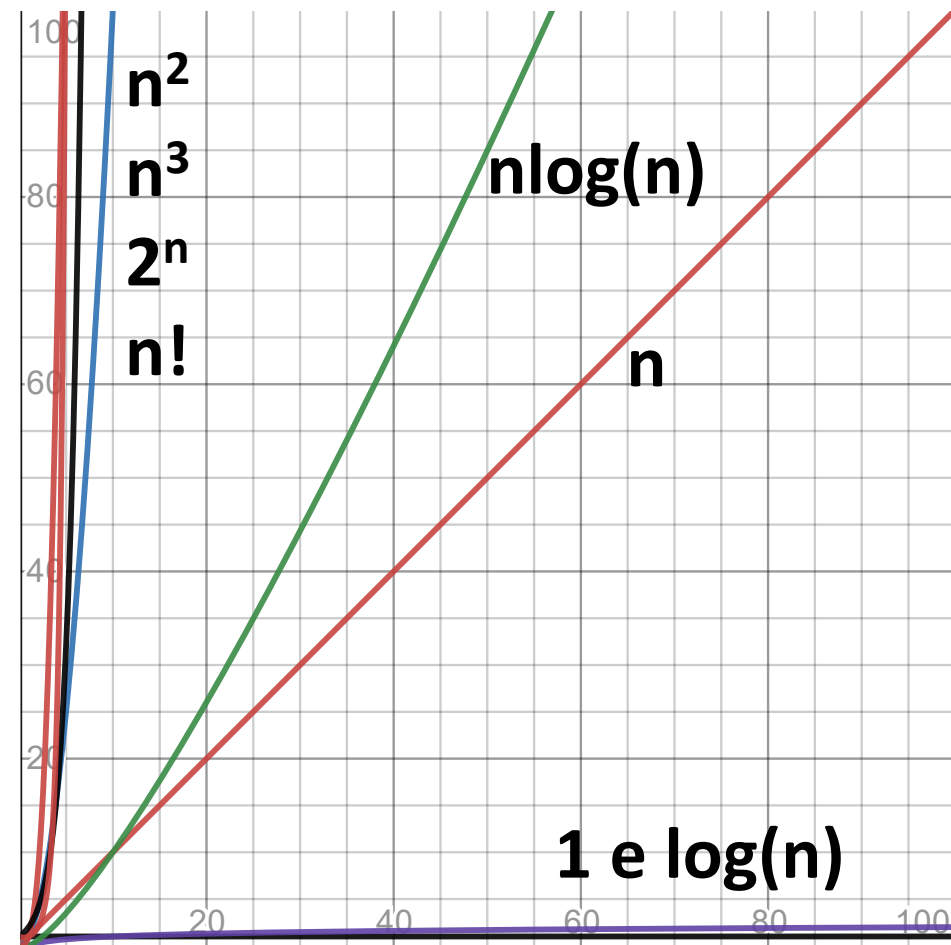
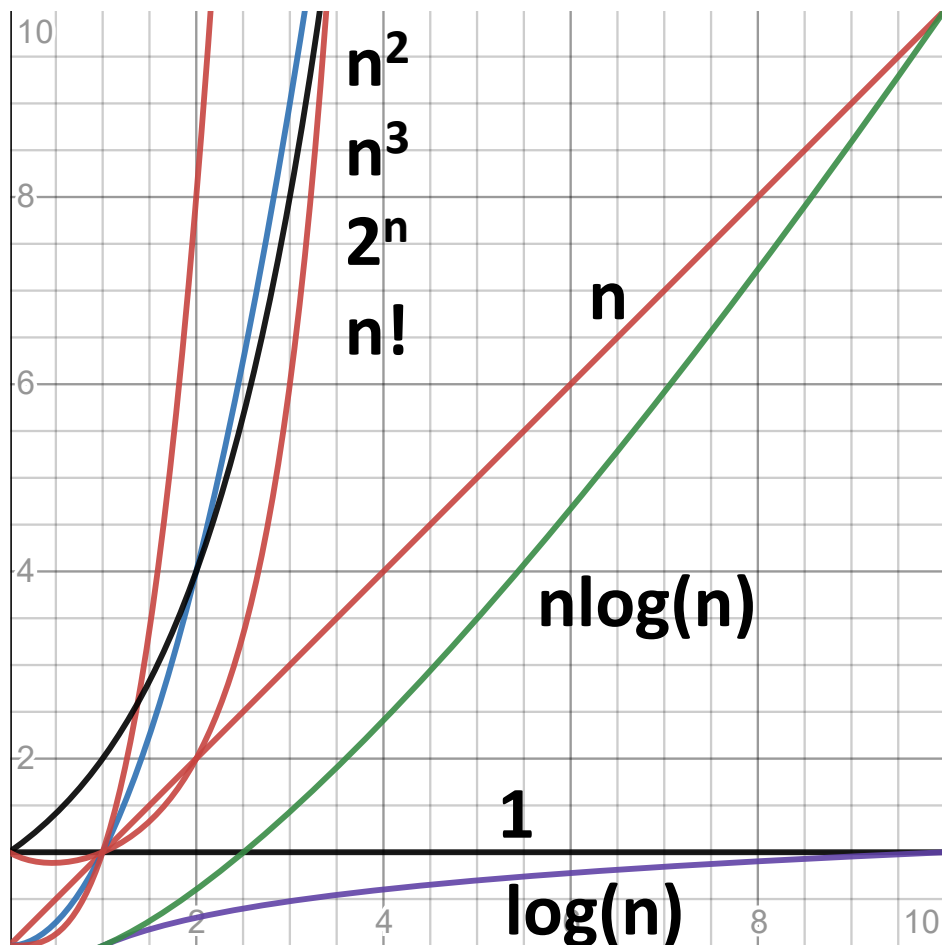
5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE

- $f(n) = O(2^n)$
 - Um algoritmo de complexidade $O(2^n)$ é dito de complexidade exponencial.
 - Geralmente não são úteis sob o ponto de vista prático.
 - Ocorrem na solução de problemas quando se usa força bruta para resolvê-los.
 - Quando n é 20, o tempo de execução é cerca de 1 milhão. Quando n dobra, o tempo fica elevado ao quadrado.

5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE

- $f(n) = O(n!)$
 - Um algoritmo de complexidade $O(n!)$ é dito de complexidade exponencial, apesar de a complexidade fatorial $O(n!)$ ter comportamento muito pior do que $O(2^n)$.
 - Geralmente ocorrem quando se usa força bruta para na solução do problema.
 - $n = 20 \rightarrow 20! = 2432902008176640000$, um número com 19 dígitos.
 - $n = 40 \rightarrow$ um número com 48 dígitos.

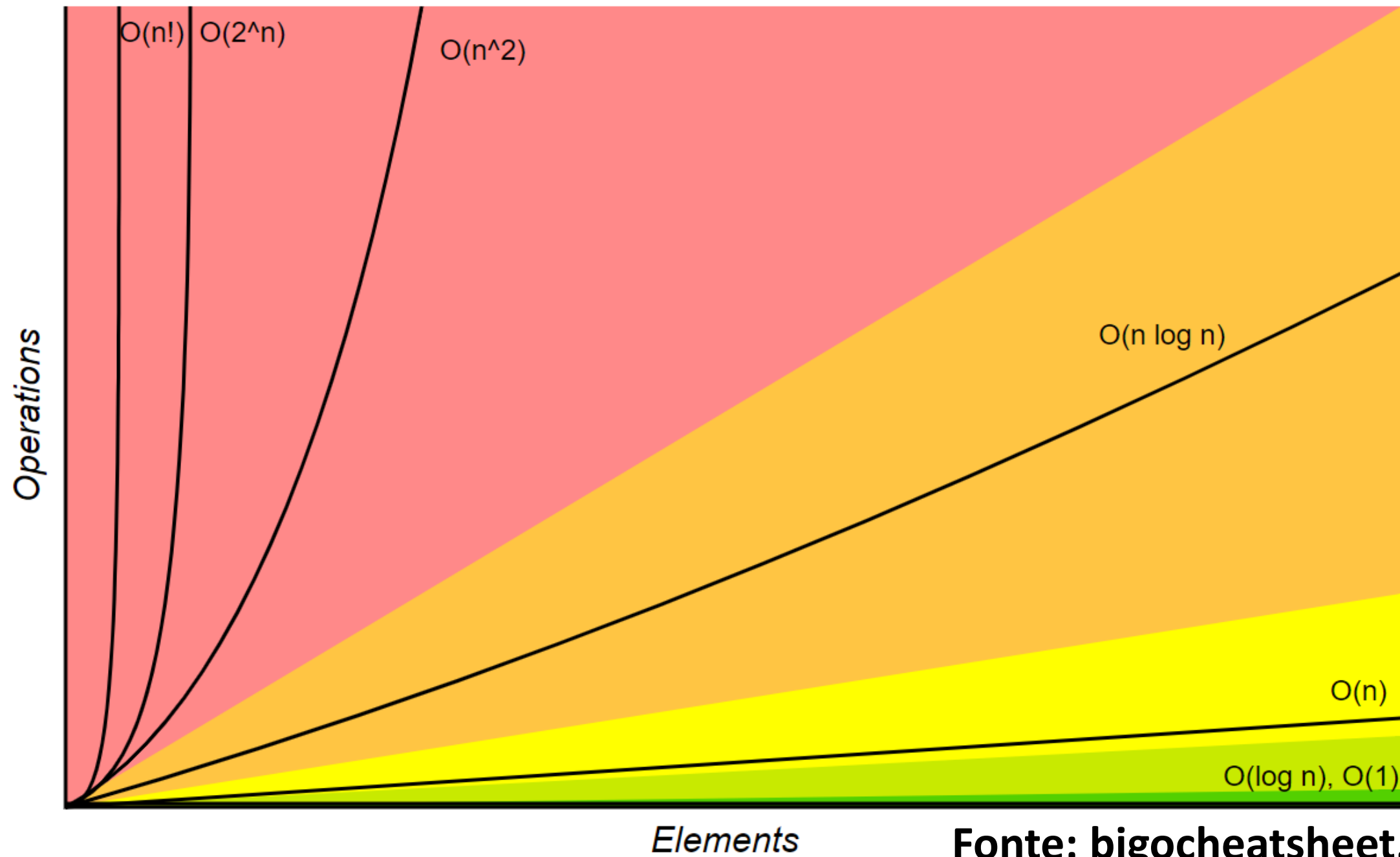
5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE



5.3 PRINCIPAIS CLASSES DE COMPLEXIDADE

Função de custo	Tamanho n					
	10	20	30	40	50	60
n	0,00001 s	0,00002 s	0,00003 s	0,00004 s	0,00005 s	0,00006 s
n^2	0,0001 s	0,0004 s	0,0009 s	0,0016 s	0,0.35 s	0,0036 s
n^3	0,001 s	0,008 s	0,027 s	0,64 s	0,125 s	0.316 s
n^5	0,1 s	3,2 s	24,3 s	1,7 min	5,2 min	13 min
2^n	0,001 s	1 s	17,9 min	12,7 dias	35,7 anos	366 séc.
3^n	0,059 s	58 min	6,5 anos	3855 séc.	10^8 séc.	10^{13} séc.

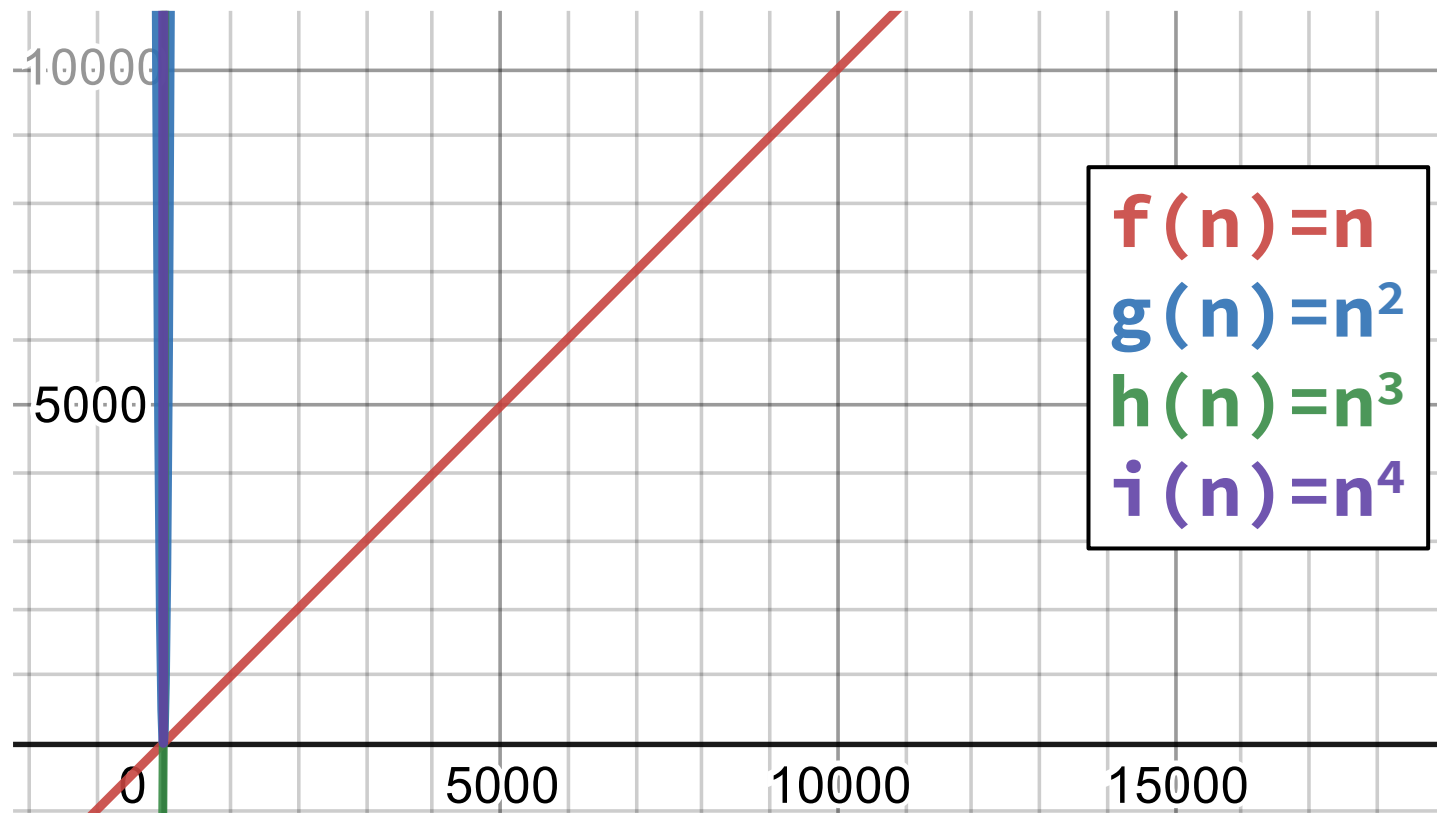
Horrible Bad Fair Good Excellent



5.4 ALGORITMOS POLINOMIAIS X EXPONENCIAIS

- Algoritmos exponenciais no tempo de execução tem função de complexidade $O(c^n)$, $c > 1$.
- Algoritmo polinomial no tempo de execução tem função de complexidade $O(p(n))$, onde $p(n)$ é um polinômio.
- A distinção entre estes dois tipos de algoritmos torna-se significativa quando o tamanho do problema a ser resolvido cresce. Por isso, os algoritmos polinomiais são muito mais úteis na prática do que os exponenciais.

5.4 ALGORITMOS POLINOMIAIS X EXPONENCIAIS



5.4 ALGORITMOS POLINOMIAIS X EXPONENCIAIS

- Algoritmos exponenciais são geralmente simples variações de pesquisa exaustiva.
- Algoritmos polinomiais são geralmente obtidos mediante entendimento mais profundo da estrutura do problema.
- Um problema é considerado:
 - intratável: se não existe um algoritmo polinomial para resolvê-lo.
 - bem resolvido: quando existe um algoritmo polinomial para resolvê-lo.

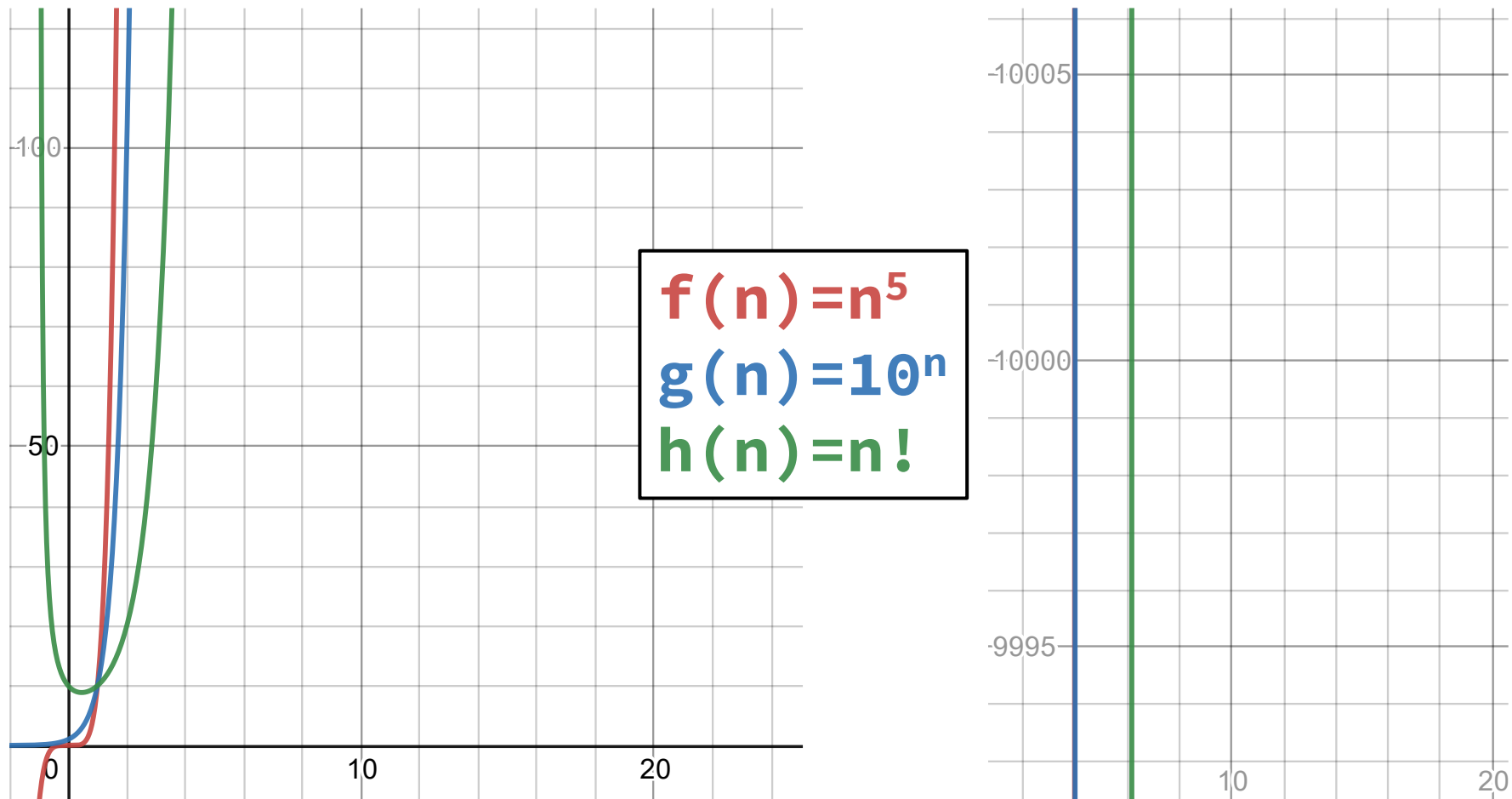
5.4 ALGORITMOS POLINOMIAIS X EXPONENCIAIS

- A distinção entre algoritmos polinomiais eficientes e algoritmos exponenciais ineficientes possui várias exceções.
- Exemplo: um algoritmo com função de complexidade $f(n) = 2^n$ é mais rápido que um algoritmo $g(n) = n^5$ para valores de n menores ou iguais a 20.

5.4 ALGORITMOS POLINOMIAIS X EXPONENCIAIS

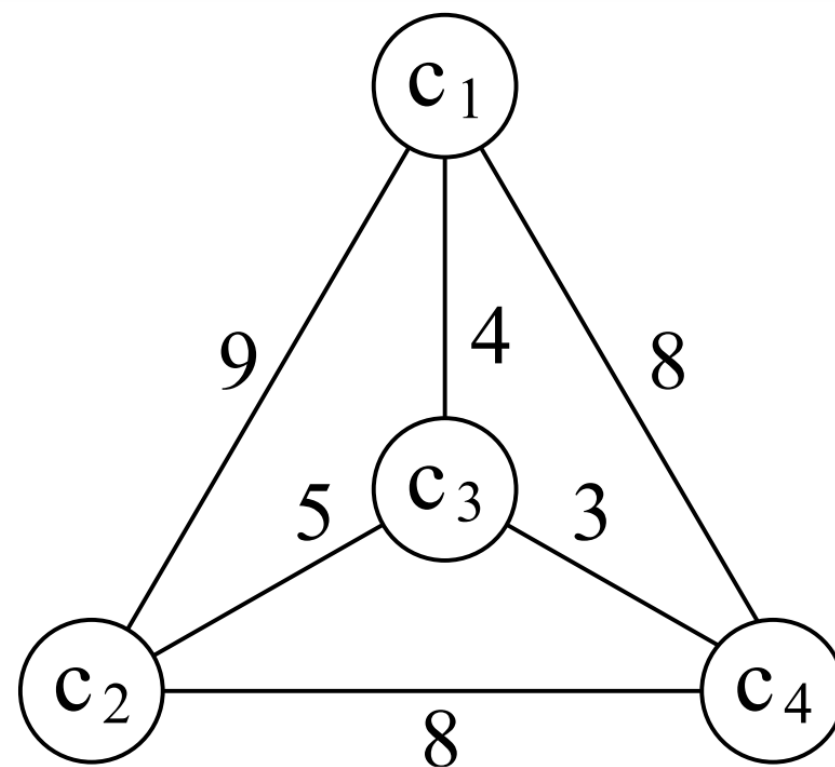
- Também existem algoritmos exponenciais que são muito úteis na prática.
 - Exemplo: o algoritmo Simplex para programação linear possui complexidade de tempo exponencial para o pior caso mas executa muito rápido na prática.
- Tais exemplos não ocorrem com frequência na prática, e muitos algoritmos exponenciais conhecidos não são muito úteis.

5.4 ALGORITMOS POLINOMIAIS X EXPONENCIAIS



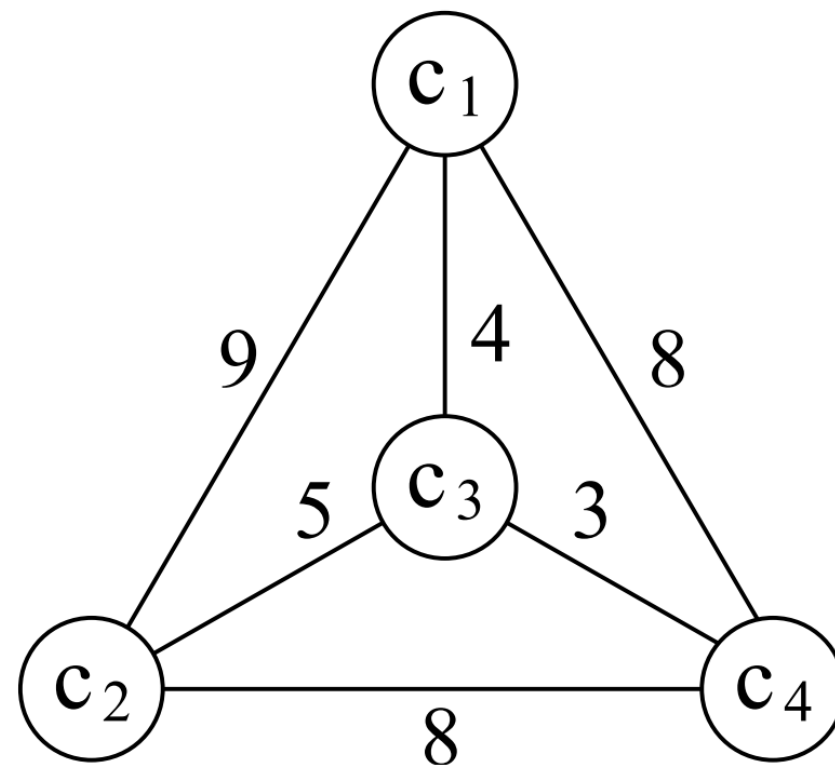
5.4 ALGORITMOS POLINOMIAIS X EXPONENCIAIS

- Exemplo de algoritmo exponencial (caixeiro viajante):
 - Um caixeiro viajante deseja visitar n cidades de tal forma que sua viagem inicie e termine em uma mesma cidade, e cada cidade deve ser visitada uma única vez.
 - Supondo que sempre há uma estrada entre duas cidades quaisquer, o problema é encontrar a menor rota para a viagem.



5.4 ALGORITMOS POLINOMIAIS X EXPONENCIAIS

- Para o exemplo ao lado, o percurso $\langle c_1, c_3, c_4, c_2, c_1 \rangle$ é uma solução para o problema, cujo percurso total tem distância 24.
- Um algoritmo simples seria verificar todas as rotas e escolher a menor delas.
- Há $(n - 1)!$ rotas possíveis e a distância total percorrida em cada rota envolve n adições, logo o número total de adições é $n!$.
- Nesse exemplo, teríamos 24 adições.



5.4 ALGORITMOS POLINOMIAIS X EXPONENCIAIS

- Suponha agora 50 cidades: o número de adições seria $50! \approx 10^{64}$.
- Em um computador que executa 10^9 adições por segundo, o tempo total para resolver o problema com 50 cidades seria maior do que 10^{45} séculos só para executar as adições.
- O problema do caixeiro viajante aparece com frequência em problemas relacionados com transporte, mas também aplicações importantes relacionadas com otimização de caminho percorrido.